

 Confidence Intervals

INSTANT

 ANOVA F-Statistic Data Analysis Diploma • Session 16

CONFIDENCE INTERVALS & ANOVA

PANDAS FUNDAMENTALS

Estimate → Compare Groups → Load Data → Analyze · ANOVA · Pandas Series ·
DataFrames · CSV / Excel / JSON / SQL

4 Parts

Stats & Pandas Core
Confidence Intervals to Data
Import

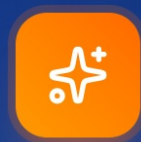
24 Slides

PDF Presentation Deck
Worked Examples & Exercises

4 Exercises

Knowledge Checks
Calculations, Selection &
Imports

 Pandas DataFrames & Series Reading CSV / Excel / JSON / SQL



01 • HOW UNCERTAIN ARE WE?

How Uncertain Are We?

Putting honest error bars around our estimates of the unknown

— MISCONCEPTION VS CORRECT MEANING

What Is a Confidence Interval?

Fix the most common misconception first: Probability is about the procedure, not any single calculated interval.

WHAT MOST PEOPLE THINK [WRONG]

"A 95% confidence interval means there is a 95% probability that the true population mean lies within this interval."

This is incorrect.

The population mean μ is a fixed (unknown) constant. It either is in your interval or it is not — there is no probability about it once the interval is calculated. The interval either captures it or misses it.

WHAT IT ACTUALLY MEANS [CORRECT]

"If we repeated our sampling procedure many times and built a confidence interval each time, 95% of those intervals would contain the true population mean."

The probability is about the PROCEDURE, not about any single interval.

Practical reading: 'We are 95% confident our method produced an interval that contains the true value.'

Visualising the Procedure (20 samples, each builds its own 95% CI):

95% capture true mean | 5% miss it

— $CI = \bar{X} \pm Z^* (\sigma / \sqrt{N})$

Confidence Interval — Formula and Full Worked Example

Customer Satisfaction Survey: $n = 100$, $\bar{x} = 7.8$, $s = 1.5$. 95% CI = (7.51, 8.09) — true average is between 7.51 and 8.09



Confidence Interval — Formula and Full Worked Example

Customer Satisfaction Survey: $n = 100$, $\bar{x} = 7.8$, $s = 1.5$. 95% CI = (7.51, 8.09) — true average is between 7.51 and 8.09

Survey $n = 100$, $\bar{x} = 7.8$, $s = 1.5$, 95% Confidence ($z^* = 1.96$)
 $SE = 1.5 / \sqrt{100} = 0.15$ | Margin of Error = $1.96 \times 0.15 = 0.294$
95% CI = $7.8 \pm 0.294 = (7.51, 8.09)$ customer satisfaction score

— COMPARING 90%, 95%, AND 99% CI

Confidence Intervals in Product Analytics

Scenario: Product team surveys 400 users about daily active usage. $\bar{x}$ = 23 min/day, s = 8 min. $SE = 0.4$.

Scenario: Your product team surveys 400 users about daily active usage. $\bar{x}$ = 23 min/day, s = 8 min. $SE = 8 / \sqrt{400} = 0.4$ min.

Confidence Level	z* Critical Value	Margin of Error	Interval (minutes)	When to Use
90%	1.645	+/- 0.66 min	(22.34, 23.66)	Use for internal dashboards where some risk is acceptable.
95%	1.960	+/- 0.78 min	(22.22, 23.78)	Standard industry choice (investor reports & product metrics).
99%	2.576	+/- 1.03 min	(21.97, 24.03)	Use for critical decisions (pricing, SLA commitments) where being wrong is costly.

Key insight: Increasing confidence always widens the interval. More certainty means less precision.

Key insight: Increasing n (sample size) always narrows the interval — the best way to improve precision.

— EXERCISE 01 · CONFIDENCE INTERVALS

Exercise — Confidence Intervals

A call centre monitors agent performance. A sample of $n = 81$ calls gives: average handling time $\bar{x} = 7.4$ minutes, sample std dev $s = 2.7$ minutes.

Q1: Calculate the Standard Error (SE) for this sample.

Hint: $SE = s / \sqrt{n}$. Use $n = 81$ and $s = 2.7$.

Reveal Solution ▾

Q2: Build a 95% confidence interval for the true mean handling time.

Hint: Use $z^ = 1.96$. $CI = \bar{x} \pm z^* \times SE$.*

Reveal Solution ▾

Q3: The target handling time is 7.0 minutes. Based on your CI, is there strong evidence that agents are exceeding the target?

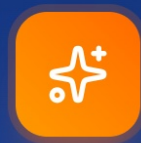
Hint: Does 7.0 fall inside or outside your interval? What does that tell you?

Reveal Solution ▾

Q4: The manager wants the margin of error reduced to 0.20 minutes at 95% confidence. What sample size is needed?

Hint: $ME = z^ \times s / \sqrt{n}$. Rearrange for n : $n = (z^* \times s / ME)^2$.*

Reveal Solution ▾



02 • COMPARING MORE THAN TWO GROUPS

Comparing More Than Two Groups

Why running many t-tests inflates your error rate — and how one test fixes it

— MULTIPLE PAIRWISE T-TESTS

Why Not Just Run Multiple T-Tests?

The hidden cost of comparing 3+ groups pairwise: Pairwise tests inflate Type I false-positive error rate.

RUNNING MANY T-TESTS

Compares only two groups at a time.

3 groups = 3 pairwise tests
5 groups = 10 pairwise tests
10 groups = 45 pairwise tests

Note: Each test risks a 5% false positive. Enough tests and a false 'significant' result becomes likely — the multiple comparisons problem.

ONE-WAY ANOVA

Tests all groups at once for any difference among them.

One test controls overall error rate
Any number of groups, one F-test
Tells you IF a difference exists, not which pair

If ANOVA finds a difference, post-hoc tests (e.g. Tukey HSD) reveal exactly which groups differ.

$$F = MSB / MSW$$

The F-Statistic — Formula and Full Worked Example

Store Region Sales: 3 store regions report average daily sales (n=5 each, N=15 total). SSB = 16,800, SSW = 14,400. $F = 7.0 > 3.89 \rightarrow$ Reject H_0 !

THE FORMULA & INPUTS

$$F = MSB / MSW \quad | \quad MSB = SSB / (k-1), \quad MSW = SSW / (N-k)$$

Groups (k):

3

Sample (N):

15

SSB:

16800

SSW:

14400

WORKED EXAMPLE — STORE REGION SALES

$k = 3$ regions, $N = 15$, $\alpha = 0.05$

$MSB = 16800 / 2 = 8400.0$

$MSW = 14400 / 12 = 1200.0$

$$F = MSB / MSW = 7.0$$

$F = 7.0 > F_{critical}(3.89) \Rightarrow$ Reject H_0

At least one region's average sales differs significantly ($p < 0.05$).

ONE-WAY ANOVA CASE STUDY

Applied Example — Does Marketing Channel Affect Customer Spend?

3 marketing channels (n=6 each, N=18): Email mean=\$85, Social mean=\$95, Search mean=\$110. $F = 4.0 > 3.68 \rightarrow$ Reject H_0 !

🌟 Applied Example — Does Marketing Channel Affect Customer Spend?

3 marketing channels (n=6 each, N=18): Email mean=\$85, Social mean=\$95, Search mean=\$110. $F = 4.0 > 3.68 \rightarrow$ Reject H_0 !

$H_0: \mu_{\text{Email}} = \mu_{\text{Social}} = \mu_{\text{Search}} \text{ (n=6 each, N=18)}$

$MSB = SSB / (3 - 1) = 2,500 / 2 = 1,250 \mid MSW = 4,687.5 / 15 = 312.5$

$F = 1,250 / 312.5 = 4.00 \text{ (df1=2, df2=15)}$

$F = 4.00 > F_{\text{critical}} (3.68) \rightarrow \text{Reject } H_0! \text{ Run Tukey HSD post-hoc test.}$

— ASSUMPTIONS & POST-HOC TESTS

ANOVA Assumptions & Post-Hoc Tests

What to check before and after running the F-test:



[IN] Independence

Observations in each group must be independent of one another.

Violated by repeated measures on same subjects.

Fix: Use Repeated Measures ANOVA for paired data.



[NM] Normality

Each group's data should be approximately normally distributed.

Check with Q-Q plot or Shapiro-Wilk test.

Fairly robust once $n > 30$ per group.



[HV] Homogeneity of Variance

Each group should have roughly equal variance (spread).

Check with Levene's test before trusting F-test.

Fix: If violated, use Welch's ANOVA.



[PH] Post-Hoc Tests

ANOVA tells you THAT groups differ, never WHICH ones.

Tukey HSD compares every pair while controlling overall error rate.

Run ONLY after a significant F-test.



[ES] Effect Size (Eta-Squared)

$\eta^2 = SSB / SST$ (% of total variance explained).

Significant F-test doesn't mean effect is practically large.

Rule of thumb: 0.01 small, 0.06 medium, 0.14 large.



[TW] Two-Way ANOVA

Tests two factors at once (e.g. Channel AND Region).

Also tests interaction: does one factor's effect depend on another?

Use when two variables jointly affect outcome.

— EXERCISE 02 · ANOVA

Exercise — ANOVA

A researcher tests 3 fertilizers on plant growth (cm), $n=5$ plants per group ($N=15$). Group means: $A=22$, $B=27$, $C=31$. $SSB=180$, $SSW=240$.

Q1: Calculate MSB and MSW.

Hint: $MSB = SSB / (k - 1)$. $MSW = SSW / (N - k)$. $k = 3$, $N = 15$.

Reveal Solution ▾

Q2: Calculate the F-statistic.

Hint: $F = MSB / MSW$.

Reveal Solution ▾

Q3: At $\alpha = 0.05$, $F_{\text{critical}}(df1=2, df2=12) = 3.89$. Is there a significant difference among fertilizers?

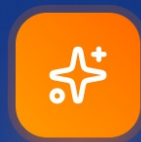
Hint: Compare your F to F_{critical} .

Reveal Solution ▾

Q4: The F-test comes back significant. Why can't you just pick the group with the highest mean and call it done?

Hint: Think about what a significant F-test does and doesn't tell you about which groups differ.

Reveal Solution ▾



03 • DATAFRAMES & SERIES

DataFrames & Series

The building blocks of data analysis in Python

SERIES VS DATAFRAME

Series vs DataFrame

pandas.Series (1D labeled array) vs pandas.DataFrame (2D labeled table of data)

PANDAS.SERIES

A single labeled, one-dimensional array of data.

One column of a spreadsheet
A labeled Python list (has an index)
`s = pd.Series([10, 20, 30])`

Goal: Hold one variable's values. Every column of a DataFrame is actually a Series under the hood.

PANDAS.DATAFRAME

A labeled, two-dimensional table of data.

A full spreadsheet or SQL table
A dict of Series sharing one index
`df = pd.DataFrame({'a':[1,2], 'b':[3,4]})`

Goal: Hold many variables together. `df['col']` returns a Series; `df[['col']]` returns a one-column DataFrame.

— CREATING & INDEXING DATAFRAMES

Creating & Indexing DataFrames

Selecting by label (.loc) vs position (.iloc) + Worked Example: Building a Sales DataFrame



Creating & Indexing DataFrames

Selecting by label (.loc) vs position (.iloc) + Worked Example: Building a Sales DataFrame

```
df = pd.DataFrame({'product': ['A', 'B', 'C'], 'region': ['North', 'South', 'North'], 'revenue': [500, 300, 700]})
Filter by Label: north = df.loc[df['region'] == 'North']
north['revenue'] returns Series values: product A: 500, product C: 700
```

— WHICH PRODUCT LINE SELLS BEST BY REGION?

Applied Example — Which Product Line Sells Best by Region?

Full pandas workflow: 01. Inspect head/info -> 02. Clean dropna -> 03. Groupby sum -> 04. idxmax top product -> 05. Business Answer

Applied Example — Which Product Line Sells Best by Region?

Full pandas workflow: 01. Inspect head/info -> 02. Clean dropna -> 03. Groupby sum -> 04. idxmax top product -> 05. Business Answer

```
1. Inspect: df.head() & df.info()
2. Clean: df = df.dropna(subset=['revenue'])
3. Group & Aggregate: grouped = df.groupby(['region', 'product'])['revenue'].sum().reset_index()
4. Top Product: top = grouped.loc[grouped.groupby('region')['revenue'].idxmax()]
Business Answer: Focus regional promotions on each region's top revenue performer!
```

COMMON PANDAS OPERATIONS

Common Pandas Operations

Most real analysis is 90% cleaning and reshaping, 10% modeling. These operations cover most of that 90%:



[FL] Filtering Rows

`df[df['age'] > 30]` keeps only rows where condition is True.
Combine conditions with `&` (and) / `|` (or).
`df.query("age > 30")` is a readable alternative.



[NC] New / Modified Columns

`df['profit'] = df['revenue'] - df['cost']` creates a column.
`df['col'].apply(func)` runs a custom function on every value.
Vectorised operations are far faster than `apply`.



[GB] GroupBy & Aggregate

`df.groupby('region')['sales'].sum()` totals sales per region.
`.agg(['mean', 'sum', 'count'])` computes several stats at once.
Classic split-apply-combine pattern.



[MG] Merging & Joining

`pd.merge(df1, df2, on='id', how='left')` combines two tables by shared key.
`how = 'left' / 'right' / 'inner' / 'outer'` controls rows.
SQL JOIN equivalent.



[MS] Missing Data

`df.isna().sum()` counts missing values per column.
`df.fillna(0)` or `df.dropna()` handles them.
Decide fill vs drop based on what missingness means.



[SR] Sorting & Ranking

`df.sort_values('revenue', ascending=False)` orders rows.
`df.nlargest(5, 'revenue')` grabs top 5 directly.
`.rank()` assigns rank within column.

— EXERCISE 03 · PANDAS DATAFRAMES

Exercise — Pandas DataFrames & Series

A DataFrame 'orders' has columns: customer_id, city, amount, status ('paid'/'refunded'), with 2,000 rows.

Q1: Select only the 'amount' and 'city' columns as a new DataFrame.

Hint: Use a list of column names inside double brackets: df[...].

Reveal Solution ▾

Q2: Filter to rows where status is 'paid' only.

Hint: df.loc[df['status'] == 'paid'].

Reveal Solution ▾

Q3: Calculate total paid revenue per city, sorted highest to lowest.

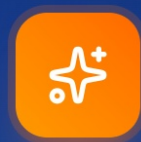
Hint: groupby('city')['amount'].sum(), then sort_values(ascending=False).

Reveal Solution ▾

Q4: 10% of the 'amount' values are missing (NaN). What should you check before deciding to drop or fill them?

Hint: Think about whether missingness is random, or tied to something like refund status.

Reveal Solution ▾



04 • GETTING REAL DATA INTO PANDAS

Getting Real Data into Pandas

CSV, Excel, JSON, and SQL — every source, one library

— ONE CONSISTENT PATTERN: `pd.read_X()`

The Four Core Readers

One consistent pattern: `pd.read_X()` always returns a DataFrame

One consistent pattern: `pd.read_X()` always returns a DataFrame from any data source!

Format	Function	Key Parameters	Example	Common Use Case
CSV	<code>pd.read_csv()</code>	<code>sep</code> , <code>header</code> , <code>dtype</code> , <code>usecols</code>	<code>pd.read_csv('data.csv')</code>	App exports, logs, spreadsheets
Excel	<code>pd.read_excel()</code>	<code>sheet_name</code> , <code>engine='openpyxl'</code>	<code>pd.read_excel('data.xlsx', sheet_name='Q3')</code>	Business reports, finance workbooks
JSON	<code>pd.read_json()</code>	<code>orient</code> , <code>lines=True</code> (JSONL)	<code>pd.read_json('data.json')</code>	API responses, semi-structured data
SQL	<code>pd.read_sql()</code>	<code>sql</code> (query), <code>con</code> (connection)	<code>pd.read_sql('SELECT * FROM orders', con)</code>	Production database queries

Key insight: Every reader accepts a `chunksize` parameter for reading huge files in batches.

— READING A MESSY CSV EXPORT

Applied Example — Reading a Messy CSV Export

Scenario: 'sales_export.csv' uses semicolons instead of commas, has 2-row header, and text dates. A first `pd.read_csv()` fails.



Applied Example — Reading a Messy CSV Export

Scenario: 'sales_export.csv' uses semicolons instead of commas, has 2-row header, and text dates. A first `pd.read_csv()` fails.

Problem: `sales_export.csv` uses semicolons, has 2-row header, and text dates.

Fix Import: `df = pd.read_csv('sales_export.csv', sep=';', skiprows=1)`

Parse Dates: `df['date'] = pd.to_datetime(df['date'])`

Result: Clean DataFrame with `datetime64[ns]` ready for analysis!

WHAT BREAKS A CLEAN IMPORT

Reading Data — Common Pitfalls

Most 'pandas bugs' are actually data-format surprises caught too late. Check for these before you analyze:



[EN] Encoding Errors

A file saved as Latin-1 or Windows-1252 raises UnicodeDecodeError under UTF-8.

Fix: `pd.read_csv(path, encoding='latin1')` or try 'utf-8-sig' for BOM files.



[DT] Wrong Dtype Inference

ZIP codes or IDs starting with 0 get silently read as integers, dropping leading zero.

Fix: Pass `dtype={'zip': str}` explicitly at read time.



[LG] Large File Memory Issues

Loading a 10GB CSV in one call can exhaust available memory.

Fix: Use `chunksize=100000` to process in batches, or read only `usecols=[...]`.



[SH] Wrong Sheet or Header Row

Excel files often have title rows, merged cells, or multiple sheets.

Fix: Set `sheet_name='Data'` and `header=2` (or `skiprows`) explicitly.



[NJ] Nested JSON Structures

`pd.read_json()` flattens poorly when values are nested dicts/lists.

Fix: Use `pd.json_normalize(data)` for nested API responses.



[SQ] Unsafe SQL Queries

Building SQL strings with f-strings/concatenation risks SQL injection.

Fix: Pass parameters separately: `pd.read_sql(query, con, params=[...])`.

— EXERCISE 04 · READING DATA

Exercise — Reading Data

You receive: 'inventory.csv' (tab-separated, UTF-8), 'budget.xlsx' (data on sheet 'FY26'), and a 50M-row 'events.json' API dump.

Q1: Write the `pd.read_csv()` call to correctly load `inventory.csv`.

Hint: What parameter controls the delimiter when it isn't a comma?

Reveal Solution ▾

Q2: Write the `pd.read_excel()` call to load the correct sheet from `budget.xlsx`.

Hint: Which parameter names a specific sheet instead of the first one?

Reveal Solution ▾

Q3: `events.json` has nested 'user': {'id':..., 'city':...} objects in every record. How do you flatten it into columns?

Hint: Which pandas function is built specifically for nested JSON records?

Reveal Solution ▾

Q4: `events.json` is far too large to load in one call on your laptop. What's your strategy?

Hint: Think about a parameter that lets you process the file in batches.

Reveal Solution ▾

INSTANT

✓ SESSION 16 COMPLETED SUCCESSFULLY

THANK YOU

Thank you for your active participation!

🎉 **Session 16 Complete!** You have mastered Confidence Intervals, ANOVA F-tests, Pandas DataFrames & Reading Data!

[↺ Restart Presentation](#)[Open Session 17: Matplotlib & Seaborn Visualizations →](#)